



AGH

**AKADEMIA GÓRNICZO-HUTNICZA
IM. STANISŁAWA STASZICA W KRAKOWIE**

STACJA NARCIARSKA PROJEKT

**Programowanie obiektowe i inżynieria
oprogramowania**

Prowadzący: dr inż. Tymoteusz Turlej

Zespół: Julia Spała 419961, Oliwier Welner-Hardt 421261,
Kacper Wiercioch 420043

Kierunek studiów: Inżynieria Mechatroniczna

Data Oddania: 25.05.2026

AGH – Akademia Górniczo-Hutnicza
Wydział Inżynierii Mechanicznej i Robotyki
Kraków, 2026

Spis treści

1	Wprowadzenie	2
2	Struktura projektu	2
2.1	Relacje między klasami	2
3	Interfejs użytkownika	3
3.1	Elementy interfejsu	3
4	Działanie aplikacji	6
4.1	Mechanizm działania wyciągów	6
4.2	System karnetów	7
5	Fragmenty kodu	7
5.1	Dziedziczenie użytkowników	7
5.2	Obsługa wyciągu	8
5.3	Wykorzystanie <code>std::unique_ptr</code>	8
5.4	Obsługa GUI – logowanie	9
5.5	Timer odświeżający GUI	10
6	Testowanie i scenariusze	10
6.1	Scenariusze testowe	10
7	Podsumowanie	13
7.1	Trudności	13
7.2	Repozytorium projektu	13

1 Wprowadzenie

Celem projektu było stworzenie aplikacji symulującej działanie stacji narciarskiej wraz z systemem zarządzania wyciągami, użytkownikami oraz sprzedażą karnetów.

Aplikacja umożliwia obsługę czterech typów użytkowników:

- administrator – zarządzanie personelem,
- kasjer – sprzedaż i konfiguracja karnetów,
- operator – zarządzanie wyciągami,
- klient – zakup i korzystanie z karnetów.

System został zaprojektowany jako aplikacja desktopowa z wykorzystaniem frameworka Qt, a logika symulacji czasu pozwala na przyspieszanie działania programu (x1–x60), co umożliwia testowanie działania systemu w różnych warunkach.

2 Struktura projektu

Aplikacja została zaprojektowana w oparciu o podejście obiektowe z wyraźnym podziałem na warstwę logiki biznesowej oraz warstwę interfejsu użytkownika.

Główne klasy projektu:

- **Stacja** – centralny obiekt zarządzający użytkownikami, wyciągami i czasem,
- **Osoba** – klasa bazowa dla użytkowników systemu,
- **Admin, Kasjer, Operator, Klient** – klasy dziedziczące,
- **Wyciąg** – reprezentacja pojedynczego wyciągu narciarskiego,
- **Wjazd** – pojedynczy przejazd klienta,
- **Cennik** – zarządzanie cenami karnetów,
- **Zegar** – symulacja czasu systemowego.

2.1 Relacje między klasami

- Stacja agreguje użytkowników i wyciągi (kompozycja),
- Osoba jest klasą bazową dla wszystkich użytkowników,
- Klient posiada Karnet,

- Wyciąg przechowuje listę aktywnych obiektów Wjazd,
- Operator zarządza obiektami Wyciąg,
- Kasjer zarządza cenami i sprzedają karnetów.

Diagram klas:

Diagram klas przedstawiający strukturę oraz zależności pomiędzy głównymi elementami systemu został umieszczony w załączniku do sprawozdania pod nazwą „*Diagram-StacjiNarciarskiej.png*”. Ze względu na czytelność dokumentu nie został on wstawiony bezpośrednio w treść raportu.

3 Interfejs użytkownika

Interfejs aplikacji został wykonany w technologii Qt Widgets. Składa się z kilku głównych paneli:

- ekran logowania,
- panel administratora,
- panel kasjera,
- panel operatora,
- panel klienta,
- panel podglądu wyciągów i osób.

GUI wykorzystuje `QStackedWidget`, który przełącza widok w zależności od roli zalogowanego użytkownika.

3.1 Elementy interfejsu

Najważniejsze elementy:

- przyciski akcji (`Qt::PushButton`),
- pola tekstowe (`QLineEdit`),
- listy i widoki tekstowe (`QTextEdit`),
- okna dialogowe (`QInputDialog`, `QMessageBox`),
- timer odświeżający stan systemu.

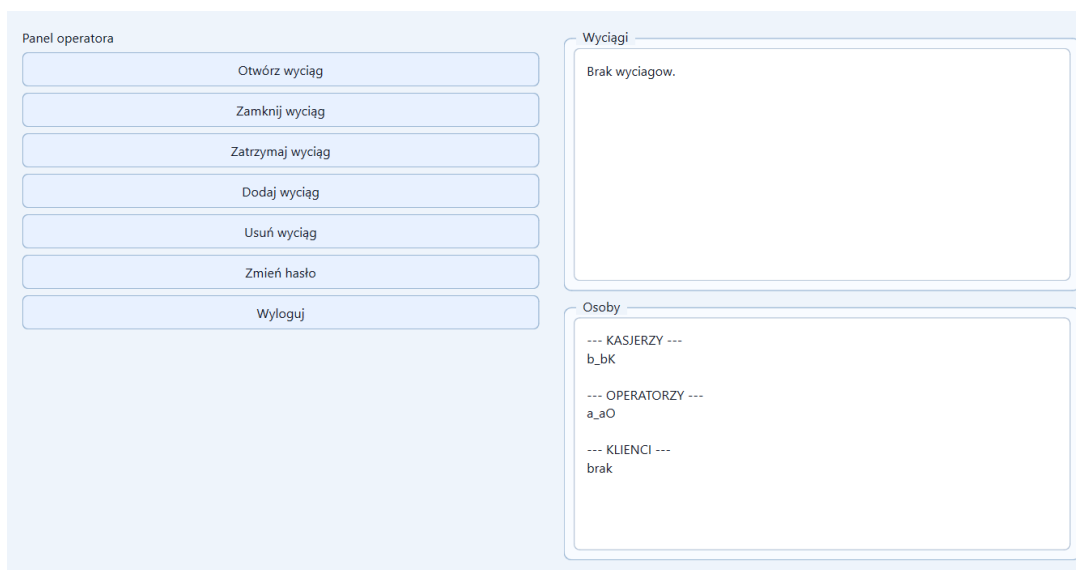
Zrzuty ekranu:

The screenshot shows the main user interface of the application. At the top, there is a header bar with the following information: **Data:** 06.12.2025, **Godzina:** 19:50:52, **Tempo czasu:** x1, and a button labeled **Ustaw datę**. Below the header, the interface is divided into two main sections. On the left, under the heading **Logowanie**, there are two input fields labeled **Login:** and **Hasło:**, followed by a **Zaloguj** button. On the right, there are two panels. The top panel, titled **Wyciągi**, displays two lines of text: **ID: 1 | otwarty | czas wjazdu: 300 s | osoby na wyciągu: 0** followed by **brak osob na wyciągu**, and **ID: 2 | zamknięty | czas wjazdu: 500 s | osoby na wyciągu: 0** followed by **brak osob na wyciągu**. The bottom panel, titled **Osoby**, displays three sections: **--- KASJERZY ---** with **b_bK**, **--- OPERATORZY ---** with **a_aO**, and **--- KLIENCI ---** with **c_c | karnet zostało: 2 dni 00:09:08** and **d_d | karnet zostało: 6 dni 00:09:08**. At the bottom of the interface, there is a status bar that reads **Status: wylogowano.**

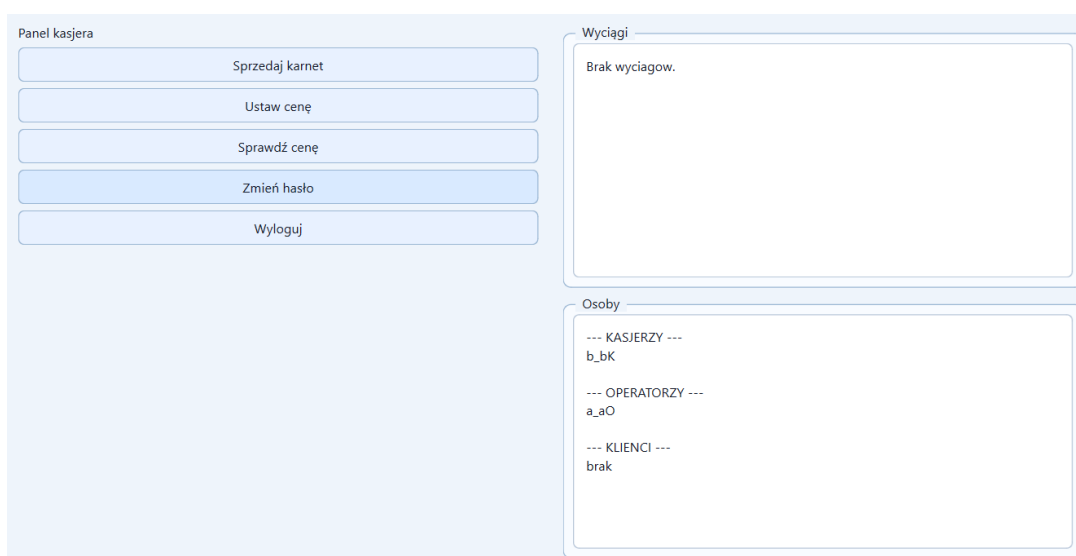
Rysunek 1: Główne okno interfejsu użytkownika aplikacji

The screenshot shows the administrator panel of the application. At the top, there is a header bar with the following information: **Data:** 06.12.2025, **Godzina:** 09:08:27, **Tempo czasu:** x1, and a button labeled **Ustaw datę**. Below the header, the interface is divided into two main sections. On the left, under the heading **Panel administratora**, there are four buttons: **Dodaj operatora**, **Dodaj kasjera**, **Zmień hasło**, and **Wyloguj**. On the right, there are two panels. The top panel, titled **Wyciągi**, displays two lines of text: **ID: 1 | otwarty | czas wjazdu: 300 s | osoby na wyciągu: 0** followed by **brak osob na wyciągu**, and **ID: 2 | zamknięty | czas wjazdu: 500 s | osoby na wyciągu: 0** followed by **brak osob na wyciągu**. The bottom panel, titled **Osoby**, displays three sections: **--- KASJERZY ---** with **b_bK**, **--- OPERATORZY ---** with **a_aO**, and **--- KLIENCI ---** with **c_c | karnet zostało: 2 dni 00:09:08** and **d_d | karnet zostało: 6 dni 00:09:08**. At the bottom of the interface, there is a status bar that reads **Status: wylogowano.**

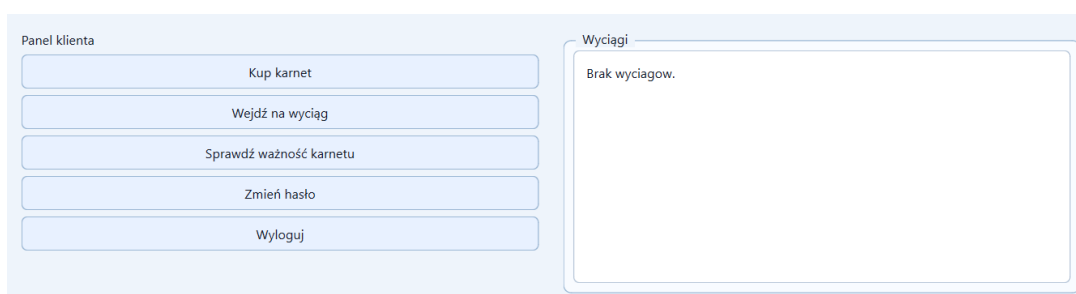
Rysunek 2: Panel administratora umożliwiający zarządzanie użytkownikami systemu.



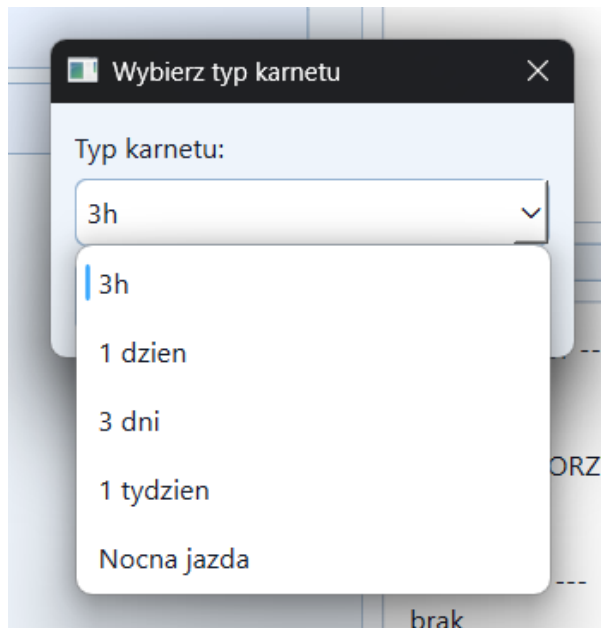
Rysunek 3: Panel operatora umożliwiający zarządzanie wyciągami i ich kontrolę.



Rysunek 4: Panel kasjera służący do sprzedaży karnetów oraz zarządzania cenami biletów.



Rysunek 5: Panel klienta umożliwiający zakup oraz sprawdzanie ważności karnetów.



Rysunek 6: Okno zakupu karnetu pozwalające na wybór typu i realizację transakcji.

4 Działanie aplikacji

Aplikacja działa w trybie symulacji czasu rzeczywistego. Zegar systemowy może być przyspieszany, co wpływa na:

- czas trwania wjazdów na wyciąg,
- ważność karnetów,
- aktywność wyciągów.

Po zalogowaniu użytkownik uzyskuje dostęp do funkcji zależnych od roli.

4.1 Mechanizm działania wyciągów

Każdy wyciąg:

- może być otwierany i zamykany przez operatora,
- może być czasowo zatrzymany,
- obsługuje listę aktywnych wjazdów,
- blokuje możliwość wejścia w przypadku stanu zamknięcia lub zatrzymania,
- uniemożliwia zamknięcie wyciągu, jeżeli znajdują się na nim aktywni użytkownicy.

4.2 System karnetów

Karnety mają różne typy czasowe (3h, 1 dzień, 3 dni, 1 tydzień, nocne). System:

- sprawdza ważność karnetu,
- automatyczna aktywacja przy pierwszym wejściu na wyciąg,
- blokuje dostęp do wyciągu po wygaśnięciu karnetu.
- ograniczenie do jednego aktywnego karnetu na osobę,

Mechanizm karnetów stanowi jeden z kluczowych elementów systemu kontroli dostępu do wyciągów. Każdy karnet posiada określony czas ważności, który jest weryfikowany w momencie próby wejścia klienta na wyciąg.

5 Fragmenty kodu

5.1 Dziedziczenie użytkowników

```
class Osoba {
public:
    virtual ~Osoba() = default;
    virtual std::string getLogin() const = 0;
};
```

Klasy **Admin**, **Kasjer**, **Operator** oraz **Klient** dziedziczą po klasie bazowej **Osoba**, co pozwala na zastosowanie mechanizmu polimorfizmu.

Dzięki temu wszystkie typy użytkowników mogą być przechowywane w jednej strukturze danych (np. w wektorze wskaźników do **Osoba**), a ich rzeczywiste zachowanie jest określone w czasie wykonania programu.

Każda z klas pochodnych implementuje własny zestaw funkcji odpowiadających jej roli w systemie:

- **Admin** – zarządzanie pracownikami systemu (dodawanie kasjerów i operatorów),
- **Kasjer** – obsługa sprzedaży i konfiguracji karnetów,
- **Operator** – zarządzanie stanem wyciągów narciarskich,
- **Klient** – zakup i wykorzystanie karnetu oraz korzystanie z wyciągów.

Zastosowanie dziedziczenia umożliwia centralne zarządzanie użytkownikami oraz ułatwia rozbudowę systemu o nowe role bez konieczności modyfikacji istniejącej logiki.

5.2 Obsługa wyciągu

```
bool Wyciag::czyMoznaWsiasc(int czas_obecny){
    aktualizujStan(czas_obecny);
    return czyOtwarty && !czyZatrzymany;
}
```

Metoda `czyMoznaWsiasc` odpowiada za kontrolę dostępu użytkownika do wyciągu w danym momencie symulacji.

Przed dokonaniem sprawdzenia wywoływana jest funkcja `aktualizujStan`, która synchronizuje stan obiektu z aktualnym czasem systemowym. Dzięki temu uwzględniane są zmiany takie jak zakończenie zatrzymania wyciągu lub zmiana jego dostępności.

Następnie sprawdzane są dwa warunki:

- czy wyciąg jest otwarty (`czyOtwarty`),
- czy nie jest aktualnie zatrzymany (`!czyZatrzymany`).

Tylko spełnienie obu warunków jednocześnie pozwala na wejście użytkownika na wyciąg. Mechanizm ten zapewnia spójność działania systemu w warunkach symulacji czasu oraz obsługę stanów dynamicznych wyciągu.

5.3 Wykorzystanie `std::unique_ptr`

W projekcie zastosowano mechanizm inteligentnych wskaźników `std::unique_ptr` w celu automatycznego zarządzania pamięcią dynamiczną oraz uniknięcia wycieków pamięci. Rozwiązanie to zostało wykorzystane między innymi do przechowywania obiektów reprezentujących wyciągi, karnety, użytkowników oraz aktywne wjazdy.

Przykładem jest klasa `Wyciag`, w której lista aktywnych wjazdów została zdefiniowana jako:

```
std::vector<std::unique_ptr<Wjazd>> wjazdy;
```

Nowe obiekty typu `Wjazd` tworzone są z wykorzystaniem funkcji `std::make_unique`:

```
wjazdy.push_back(
    std::make_unique<Wjazd>(
        czas_obecny,
        czas_wjazdu,
        klientLogin
    )
);
```

Podobne rozwiązanie zastosowano również w klasie `Stacja`, gdzie kolekcje użytkowników i wyciągów przechowywane są w postaci inteligentnych wskaźników:

```
std::vector<std::unique_ptr<Wyciag>> wyciagi;
std::vector<std::unique_ptr<Osoba>> osoby;
std::vector<std::unique_ptr<Osoba>> pracownicy;
```

Dodawanie nowych obiektów odbywa się poprzez:

```
stacja.pracownicy.push_back(
    std::make_unique<Admin>(
        "Jan",
        "Kowalski",
        "administrator",
        "haslo"
    )
);
```

Zastosowanie `std::unique_ptr` pozwoliło uprościć zarządzanie cyklem życia obiektów oraz zwiększyć bezpieczeństwo działania aplikacji.

5.4 Obsługa GUI – logowanie

```
void MainWindow::onLoginClicked()
{
    aktualnyUzytkownik = stacja.zaloguj(login, haslo);

    if (dynamic_cast<Admin*>(aktualnyUzytkownik)) {
        ui->pagesWidget->setCurrentIndex(1);
    }
}
```

Metoda `onLoginClicked()` odpowiada za obsługę procesu logowania użytkownika do systemu.

Po wpisaniu danych logowania następuje wywołanie metody `stacja.zaloguj()`, która zwraca wskaźnik do obiektu klasy bazowej `Osoba`. Następnie wykonywane jest sprawdzenie rzeczywistego typu obiektu za pomocą `dynamic_cast`.

W przedstawionym fragmencie sprawdzany jest przypadek użytkownika typu `Admin`. Jeśli rzutowanie zakończy się powodzeniem, oznacza to, że zalogowany użytkownik posiada uprawnienia administratora i aplikacja przełącza widok `QStackedWidget` na panel administratora (indeks 1).

Takie rozwiązanie pozwala na:

- dynamiczne rozpoznawanie ról użytkowników w czasie działania programu,
- centralizację logiki logowania w jednym miejscu,
- łatwe rozszerzenie systemu o kolejne role użytkowników.

5.5 Timer odświeżający GUI

```
refreshTimer = new QTimer(this);

connect(refreshTimer, &QTimer::timeout, this, [this]() {
    odswiezCzas();
    odswiezWyciagi();
    odswiezOsoby();
});
```

Mechanizm oparty na klasie `QTimer` odpowiada za cykliczne odświeżanie interfejsu użytkownika. Timer działa w trybie zdarzeniowym i co określony interwał czasowy emituje sygnał `timeout()`.

W przedstawionym fragmencie sygnał ten jest połączony z funkcją lambda, która wywołuje zestaw metod odpowiedzialnych za aktualizację widoku:

- `odswiezCzas()` – aktualizacja zegara systemowego,
- `odswiezWyciagi()` – odświeżenie stanu wyciągów,
- `odswiezOsoby()` – aktualizacja listy użytkowników.

Zastosowanie lambdy pozwala na uproszczenie kodu poprzez uniknięcie definiowania osobnej metody slotu, a jednocześnie umożliwia dostęp do kontekstu obiektu (`this`).

Dzięki temu GUI pozostaje aktualne bez potrzeby ręcznego odświeżania przez użytkownika, co poprawia ergonomię i responsywność aplikacji.

6 Testowanie i scenariusze

Testowanie aplikacji przeprowadzono manualnie poprzez symulację użytkowników oraz sprawdzanie różnych scenariuszy działania systemu.

6.1 Scenariusze testowe

1. Logowanie użytkownika

- **Poprawne dane logowania**

- **Stan początkowy:** użytkownik znajduje się na ekranie logowania,
- **Dane wejściowe:** poprawny login i hasło,
- **Działanie:** kliknięcie przycisku „Zaloguj”,
- **Oczekiwany wynik:** przejście do odpowiedniego panelu użytkownika,
- **Wynik rzeczywisty:** użytkownik zostaje zalogowany i widzi panel zgodny z rolą.

- **Błędne dane logowania**

- **Stan początkowy:** ekran logowania aktywny,
- **Dane wejściowe:** nieprawidłowy login lub hasło,
- **Działanie:** próba logowania,
- **Oczekiwany wynik:** odmowa dostępu oraz komunikat o błędzie,
- **Wynik rzeczywisty:** system nie loguje użytkownika i wyświetla informację o błędnych danych.

2. Zakup karnetu

- **Zakup przez klienta bez aktywnego karnetu**

- **Stan początkowy:** klient zalogowany, brak aktywnego karnetu,
- **Dane wejściowe:** wybór typu karnetu,
- **Działanie:** kliknięcie „Kup karnet”,
- **Oczekiwany wynik:** przypisanie nowego karnetu do użytkownika,
- **Wynik rzeczywisty:** karnet zostaje poprawnie aktywowany.

- **Zakup przez klienta z aktywnym karnetem**

- **Stan początkowy:** klient posiada aktywny karnet,
- **Dane wejściowe:** próba zakupu nowego karnetu,
- **Działanie:** kliknięcie „Kup karnet”,
- **Oczekiwany wynik:** blokada operacji,
- **Wynik rzeczywisty:** system uniemożliwia zakup nowego karnetu.

- **Zakup karnetu przez kasjera**

- **Stan początkowy:** kasjer zalogowany w systemie,
- **Dane wejściowe:** wybór klienta oraz typu karnetu,
- **Działanie:** operacja „Sprzedaj karnet” w panelu kasjera,
- **Oczekiwany wynik:** przypisanie karnetu wskazanemu klientowi,
- **Wynik rzeczywisty:** karnet zostaje poprawnie zapisany w systemie.

3. Wejście na wyciąg

- **Wejście na wyciąg przy stanie otwartym i aktywnym**
 - **Stan początkowy:** wyciąg otwarty i działa normalnie,
 - **Dane wejściowe:** brak dodatkowych parametrów (próba wejścia użytkownika),
 - **Działanie:** użytkownik próbuje wejść na wyciąg,
 - **Oczekiwany wynik:** pozwolenie na wejście na wyciąg,
 - **Wynik rzeczywisty:** użytkownik zostaje wpuszczony na wyciąg.
- **Wejście na wyciąg przy stanie zamkniętym lub zatrzymanym**
 - **Stan początkowy:** wyciąg zamknięty lub zatrzymany,
 - **Dane wejściowe:** próba wejścia użytkownika,
 - **Działanie:** użytkownik próbuje wejść na wyciąg,
 - **Oczekiwany wynik:** blokada wejścia,
 - **Wynik rzeczywisty:** system uniemożliwia wejście na wyciąg.

4. Operator wyciągu

- **Zamykanie pustego wyciągu**
 - **Stan początkowy:** wyciąg otwarty, brak aktywnych użytkowników,
 - **Dane wejściowe:** brak dodatkowych parametrów,
 - **Działanie:** wybór opcji „Zamknij wyciąg”,
 - **Oczekiwany wynik:** zmiana stanu wyciągu na „zamknięty”,
 - **Wynik rzeczywisty:** wyciąg zostaje poprawnie zamknięty.
- **Zamykanie wyciągu z aktywnymi użytkownikami**
 - **Stan początkowy:** wyciąg otwarty, na trasie znajdują się użytkownicy,
 - **Dane wejściowe:** brak,
 - **Działanie:** próba zamknięcia wyciągu,
 - **Oczekiwany wynik:** blokada operacji,
 - **Wynik rzeczywisty:** system uniemożliwia zamknięcie i utrzymuje stan „otwarty”.
- **Zatrzymanie awaryjne**
 - **Stan początkowy:** wyciąg pracuje normalnie,
 - **Dane wejściowe:** brak,
 - **Działanie:** wybór opcji „Zatrzymaj wyciąg”,
 - **Oczekiwany wynik:** natychmiastowe zatrzymanie pracy wyciągu,
 - **Wynik rzeczywisty:** wyciąg przechodzi w stan awaryjny i zostaje zatrzymany.

7 Podsumowanie

Projekt pozwolił na praktyczne zastosowanie programowania obiektowego oraz frameworka Qt do budowy interfejsu graficznego.

Zastosowanie symulacji czasu umożliwiło testowanie działania systemu w przyspieszonym tempie, co znacząco ułatwia analizę logiki aplikacji.

Możliwe dalsze rozszerzenia:

- implementacja bazy danych,
- system rezerwacji online,
- mapa wyciągów,
- system statystyk i raportów,
- równoległa obsługa wielu klientów,
- integracja z API pogodowym.

7.1 Trudności

W trakcie realizacji projektu napotkano trudności związane z zarządzaniem pamięcią dynamiczną. W pierwszej wersji aplikacji zastosowano standardowe wskaźniki w połączeniu z kontenerami `std::vector`, co prowadziło do problemów takich jak wycieki pamięci oraz niepoprawne usuwanie elementów z kolekcji (m.in. błędy związane z ręcznym zwalnianiem pamięci oraz niekontrolowanym cyklem życia obiektów).

Problemy te wynikały z konieczności ręcznego zarządzania obiektami oraz ryzyka podwójnego usunięcia lub pozostawienia niezwolnionej pamięci.

Rozwiązaniem okazało się zastosowanie inteligentnych wskaźników `std::weak_ptr`, które automatyzują zarządzanie pamięcią i eliminują konieczność ręcznego wywoływania operacji usuwania obiektów. Po wprowadzeniu tego mechanizmu stabilność aplikacji znacząco się poprawiła, a struktura kodu stała się bardziej bezpieczna i czytelna.

Dodatkowo warto zaznaczyć, że pierwsza wersja logiki programu była tworzona i testowana w środowisku konsolowym. Dopiero w późniejszym etapie konieczne było dostosowanie oraz rozbudowa kodu w celu integracji z graficznym interfejsem użytkownika opartym o bibliotekę Qt, co wymagało modyfikacji części struktur oraz sposobu komunikacji między modułami aplikacji.

7.2 Repozytorium projektu

Kod źródłowy oraz pełna dokumentacja projektu zostały udostępnione w publicznym repozytorium GitHub:

- **Link do repozytorium:** https://github.com/Oliwier421/projektobiektowe_git

Repozytorium zawiera kompletny kod aplikacji, pliki projektu Qt, diagramy UML oraz pozostałe materiały wykorzystane podczas realizacji systemu.